

```
1: /**
2:  * @file biblioteca.c
3:  * @brief Ordenamiento recursivo de libros con conteo de intercambios
4:  * @author Asistente IA
5:  * @date 2026
6:  *
7:  * Este programa ordena alfabéticamente una lista de títulos de libros
8:  * usando Quicksort recursivo y cuenta el número total de intercambios.
9:  */
10:
11: #include <stdio.h>
12: #include <string.h>
13:
14: #define MAX_TITULO_LEN 100
15: #define MAX_LIBROS 100
16:
17: /**
18:  * @brief Intercambia dos cadenas en un arreglo
19:  *
20:  * @param arr Arreglo de cadenas
21:  * @param i Índice del primer elemento
22:  * @param j Índice del segundo elemento
23:  */
24: void intercambiar(char *arr[], int i, int j) {
25:     if (i == j) {
26:         return; // No es necesario intercambiar consigo mismo
27:     }
28:
29:     char *temp = arr[i];
30:     arr[i] = arr[j];
31:     arr[j] = temp;
32: }
33:
34: /**
35:  * @brief Función de partición para Quicksort
36:  *
37:  * Toma el último elemento como pivote y coloca todos los elementos menores
38:  * a la izquierda y mayores a la derecha.
39:  *
40:  * @param arr Arreglo de cadenas
41:  * @param inicio Índice inicial del segmento
42:  * @param fin Índice final del segmento
43:  * @param intercambios Puntero al contador de intercambios
44:  * @return int Índice de la posición final del pivote
45:  *
46:  * @note Esta función NO usa break ni continue
47:  * @note Compara cadenas usando strcmp (orden alfabético)
48:  */
49: int particionar(char *arr[], int inicio, int fin, int *intercambios) {
50:     // Elegir el pivote como el último elemento
51:     char *pivote = arr[fin];
52:
53:     // Índice del elemento más pequeño (indica la posición donde irán los
54:     // menores)
55:     int i = inicio - 1;
56:
57:     // Recorrer el segmento [inicio, fin-1]
58:     // Usamos un bucle for estándar, pero sin break/continue
59:     for (int j = inicio; j <= fin - 1; j++) {
60:         // Si el elemento actual es menor o igual al pivote (orden alfabético)
61:         if (strcmp(arr[j], pivote) <= 0) {
62:             // Incrementar el índice del menor
63:             i++;
64:
65:             // Intercambiar arr[i] y arr[j]
66:             if (i != j) {
67:                 intercambiar(arr, i, j);
68:                 (*intercambios)++; // Contar este intercambio
69:             }
70:         }
71:     }
72:
73:     // Intercambiar el pivote con el elemento en la posición i
74:     intercambiar(arr, i, fin);
75:     (*intercambios)++; // Contar este intercambio
76:
77:     return i;
78: }
```

```
70:          // No se usa continue, simplemente se continúa con la siguiente iteración
n
71:      }
72:
73:      // Colocar el pivote en su posición correcta
74:      if (i + 1 != fin) {
75:          intercambiar(arr, i + 1, fin);
76:          (*intercambios)++; // Contar el intercambio del pivote
77:      }
78:
79:      // Retornar la posición del pivote
80:      return i + 1;
81: }
82:
83: /**
84:  * @brief Función recursiva principal de Quicksort
85:  *
86:  * @param arr Arreglo de cadenas a ordenar
87:  * @param inicio Índice inicial del segmento a ordenar
88:  * @param fin Índice final del segmento a ordenar
89:  * @param intercambios Puntero al contador de intercambios
90:  *
91:  * @details Algoritmo:
92:  *      1. Si el segmento tiene más de 1 elemento
93:  *      2. Particionar el segmento
94:  *      3. Ordenar recursivamente la parte izquierda
95:  *      4. Ordenar recursivamente la parte derecha
96:  */
97: void quicksortRecursivo(char *arr[], int inicio, int fin, int *intercambios) {
98:     // === CASO BASE ===
99:     // Si el segmento tiene 0 o 1 elemento, ya está ordenado
100:    if (inicio >= fin) {
101:        return;
102:    }
103:
104:    // === CASO RECURSIVO ===
105:    // Particionar el arreglo y obtener la posición del pivote
106:    int pivoteIndex = particionar(arr, inicio, fin, intercambios);
107:
108:    // Ordenar recursivamente los elementos izquierdos (menores que el pivote)
109:    quicksortRecursivo(arr, inicio, pivoteIndex - 1, intercambios);
110:
111:    // Ordenar recursivamente los elementos derechos (mayores que el pivote)
112:    quicksortRecursivo(arr, pivoteIndex + 1, fin, intercambios);
113: }
114:
115: /**
116:  * @brief Función wrapper para ordenar la biblioteca
117:  *
118:  * @param libros Arreglo de títulos de libros
119:  * @param numLibros Número total de libros
120:  * @param intercambios Puntero al contador de intercambios
121:  */
122: void ordenarBiblioteca(char *libros[], int numLibros, int *intercambios) {
123:     if (numLibros <= 1) {
124:         return;
125:     }
126:
127:     quicksortRecursivo(libros, 0, numLibros - 1, intercambios);
128: }
129:
130: /**
131:  * @brief Imprime el contenido de la biblioteca
132:  *
133:  * @param libros Arreglo de títulos
134:  * @param numLibros Número de libros
135:  */
136: void imprimirBiblioteca(char *libros[], int numLibros) {
137:     printf("[");
138:     for (int i = 0; i < numLibros; i++) {
```

```
139:     printf("\n%s", libros[i]);
140:     if (i < numLibros - 1) {
141:         printf(" ");
142:     }
143: }
144: printf("]\n");
145: }
146:
147: /**
148:  * @brief Versi3n con depuraci3n paso a paso
149:  *
150:  * Muestra el estado del arreglo despu3s de cada partici3n
151:  * para entender c3mo funciona la recursi3n.
152:  *
153:  * @param libros Arreglo de t3tulos
154:  * @param inicio Inicio del segmento
155:  * @param fin Fin del segmento
156:  * @param profundidad Nivel de recursi3n (para indentaci3n)
157:  * @param intercambios Puntero al contador
158:  * @return int 3ndice del pivote
159:  */
160: int particionarDebug(char *arr[], int inicio, int fin,
161:                     int profundidad, int *intercambios) {
162:
163:     // Imprimir indentaci3n
164:     for (int i = 0; i < profundidad; i++) {
165:         printf(" ");
166:     }
167:
168:     char *pivote = arr[fin];
169:     printf("Partici3n [%d..%d], pivote=\"%s\"\n", inicio, fin, pivote);
170:
171:     int i = inicio - 1;
172:
173:     for (int j = inicio; j <= fin - 1; j++) {
174:         for (int k = 0; k < profundidad; k++) {
175:             printf(" ");
176:         }
177:         printf(" Comparando \"%s\" con \"%s\"", arr[j], pivote);
178:
179:         if (strcmp(arr[j], pivote) <= 0) {
180:             i++;
181:             if (i != j) {
182:                 printf(" 3\222 intercambiar (menor o igual)");
183:                 intercambiar(arr, i, j);
184:                 (*intercambios)++;
185:
186:                 // Mostrar estado actual
187:                 printf(" 3\222 ");
188:                 for (int k = inicio; k <= fin; k++) {
189:                     printf("\n%s", arr[k]);
190:                 }
191:             } else {
192:                 printf(" 3\222 ya est3 en posici3n");
193:             }
194:         } else {
195:             printf(" 3\222 es mayor, queda donde est3");
196:         }
197:         printf("\n");
198:     }
199:
200:     // Colocar pivote en posici3n correcta
201:     if (i + 1 != fin) {
202:         for (int k = 0; k < profundidad; k++) {
203:             printf(" ");
204:         }
205:         printf(" Moviendo pivote a posici3n %d\n", i + 1);
206:         intercambiar(arr, i + 1, fin);
207:         (*intercambios)++;
208:     }
```

```
209:
210:     return i + 1;
211: }
212:
213: /**
214:  * @brief Versi3n de depuraci3n de Quicksort
215:  */
216: void quicksortDebug(char *arr[], int inicio, int fin,
217:                     int profundidad, int *intercambios) {
218:
219:     // Indentaci3n
220:     for (int i = 0; i < profundidad; i++) {
221:         printf(" ");
222:     }
223:     printf("Quicksort [%d..%d]\n", inicio, fin);
224:
225:     // Caso base
226:     if (inicio >= fin) {
227:         for (int i = 0; i < profundidad; i++) {
228:             printf(" ");
229:         }
230:         printf("222 Segmento trivial, retornar\n");
231:         return;
232:     }
233:
234:     int pivotIndex = particionarDebug(arr, inicio, fin, profundidad, intercambios
);
235:
236:     for (int i = 0; i < profundidad; i++) {
237:         printf(" ");
238:     }
239:     printf("Estado actual: ");
240:     for (int i = inicio; i <= fin; i++) {
241:         printf("\%s", arr[i]);
242:     }
243:     printf("\n");
244:
245:     quicksortDebug(arr, inicio, pivotIndex - 1, profundidad + 1, intercambios);
246:     quicksortDebug(arr, pivotIndex + 1, fin, profundidad + 1, intercambios);
247: }
248:
249: /**
250:  * @brief Funci3n principal
251:  */
252: int main() {
253:     printf("=== ORDENAMIENTO DE BIBLIOTECA ===\n");
254:     printf("M3todo: Quicksort recursivo\n\n");
255:
256:     // === CASO DE PRUEBA PROPORCIONADO ===
257:     char *libros1[] = {"Zoo", "Antolog3a", "Bibliograf3a", "Cient3fica", "Drama
"};
258:     int numLibros1 = sizeof(libros1) / sizeof(libros1[0]);
259:
260:     printf("Entrada:\n");
261:     imprimirBiblioteca(libros1, numLibros1);
262:
263:     // Crear copia para no modificar el original
264:     char *copial[MAX_LIBROS];
265:     for (int i = 0; i < numLibros1; i++) {
266:         copial[i] = libros1[i];
267:     }
268:
269:     int intercambios1 = 0;
270:     ordenarBiblioteca(copial, numLibros1, &intercambios1);
271:
272:     printf("\nSalida:\n");
273:     imprimirBiblioteca(copial, numLibros1);
274:     printf("234223 Se ha podido ordenar el librero en %d pasos (intercambios)\n
",
275:           intercambios1);
```

```
276:
277: // === VERSIÃ\223N CON DEPURACIÃ\223N ===
278: printf("=== PROCESO DETALLADO DE ORDENAMIENTO ===\n\n");
279:
280: char *libros2[] = {"Zoo", "AntologÃ-a", "BibliografÃ-a", "CientÃ-fica", "Drama
"};
281: int numLibros2 = sizeof(libros2) / sizeof(libros2[0]);
282:
283: char *copia2[MAX_LIBROS];
284: for (int i = 0; i < numLibros2; i++) {
285:     copia2[i] = libros2[i];
286: }
287:
288: int intercambios2 = 0;
289: quicksortDebug(copia2, 0, numLibros2 - 1, 0, &intercambios2);
290:
291: printf("\nResultado final: ");
292: imprimirBiblioteca(copia2, numLibros2);
293: printf("Total de intercambios: %d\n\n", intercambios2);
294:
295: // === PRUEBAS ADICIONALES ===
296: printf("=== PRUEBAS ADICIONALES ===\n");
297:
298: // Prueba 1: Lista ya ordenada
299: char *prueba1[] = {"A", "B", "C", "D"};
300: int n1 = sizeof(prueba1) / sizeof(prueba1[0]);
301: char *copiaP1[MAX_LIBROS];
302: for (int i = 0; i < n1; i++) copiaP1[i] = prueba1[i];
303: int inter1 = 0;
304: ordenarBiblioteca(copiaP1, n1, &inter1);
305: printf("\nLista ordenada [A,B,C,D]: %d intercambios (mÃ-nimo)\n", inter1);
306:
307: // Prueba 2: Lista inversa
308: char *prueba2[] = {"D", "C", "B", "A"};
309: int n2 = sizeof(prueba2) / sizeof(prueba2[0]);
310: char *copiaP2[MAX_LIBROS];
311: for (int i = 0; i < n2; i++) copiaP2[i] = prueba2[i];
312: int inter2 = 0;
313: ordenarBiblioteca(copiaP2, n2, &inter2);
314: printf("Lista inversa [D,C,B,A]: %d intercambios\n", inter2);
315:
316: // Prueba 3: Libro repetido
317: char *prueba3[] = {"MatemÃ;ticas", "Arte", "MatemÃ;ticas", "FÃ-sica"};
318: int n3 = sizeof(prueba3) / sizeof(prueba3[0]);
319: char *copiaP3[MAX_LIBROS];
320: for (int i = 0; i < n3; i++) copiaP3[i] = prueba3[i];
321: int inter3 = 0;
322: ordenarBiblioteca(copiaP3, n3, &inter3);
323: printf("Con repeticiones: ");
324: imprimirBiblioteca(copiaP3, n3);
325: printf(" Intercambios: %d\n", inter3);
326:
327: return 0;
328: }
```